claude-windows / e83b09d8-e113-4d4d-9411-cd80ae217af9

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\subagents\workflows\wf_ae339437-da9\agent-ab817e3361799239f.jsonl`
- SHA-256: `504ed9edfd75382c8e1cc6afedbeb236e4dc00e73d7a9223b6630510183ce186`
- Source modified: `2026-07-07T15:52:13+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf_ae339437-da9`
- session_id: `e83b09d8-e113-4d4d-9411-cd80ae217af9`

Transcript

1. user (2026-07-07T15:48:13.158Z)

```
You are reviewing ONE commit on branch `fix/cloud-run-exec-status-fastfail` in the repo at:
  C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
Always start by: cd "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer" (the
Bash cwd may default elsewhere). Use Read/Grep with absolute paths under that repo.

The commit hardens `CloudRunCompute.run_infer` (backend/compute/cloud_run.py): the done-marker
stays the
SOLE success signal, but each bucket-poll ALSO probes the Cloud Run Executions API so a worker
that
TERMINATED without ever writing a marker (crash/OOM/an old image argparse-aborting on an unknown
flag)
fails fast (raise RemoteExecutionFailed) instead of polling the full remote_poll_max_wait_sec
(~65 min).
`_dispatch_remote` maps RemoteExecutionFailed -> rc 5 (distinct from the rc 4 poll-timeout).

CRITICAL CONTEXT: this commit was REBASED onto a master that had since merged two sibling PRs
touching
the same files:
  - #513 (23c1868): added `ComputeJobSpec.skip_validation` + a `--skip-validation` worker flag;
the
    control-plane dispatcher sets it so the worker skips redundant re-validation.
  - #512 (20862d7): a validators shared-decode refactor (file-disjoint from this commit).
The motivating scenario for THIS fix: a post-#513 control-plane dispatching `--skip-validation`
to an
OLD worker image (no such flag) -> the worker argparse-aborts (exit 2) BEFORE writing a marker
-> the
exact ~65-min hang this fix collapses to a prompt rc-5 failure.

To see EXACTLY what this commit changes (only 5 files), run:
  git diff origin/master...HEAD        (three-dot: my change vs merge-base)
To see the sibling context that landed on the same files, run:
  git diff 3be0935..origin/master -- backend/compute/base.py backend/compute/cloud_run.py
backend/worker/__main__.py

Read the FULL current files (not just the diff) to judge correctness in context:
  backend/compute/cloud_run.py, backend/compute/base.py, backend/compute/__init__.py,
  backend/worker/__main__.py, backend/orchestration.py (the API caller),
tests/test_compute_backend.py,
  backend/infrastructure/execution_policy.py (poll_until semantics).

Report ONLY real, defect-grade findings with a concrete failure scenario. Do NOT report style
nits,
things the gate already covers, or speculation. If the change is correct on your lens, return an
```

```
empty
findings array and say so in `assessment`. Full suite (1833) + ruff + mypy + coverage 85.66%
already pass;
do not re-report "add a test" unless a SPECIFIC untested behavior would plausibly break.

LENS ? REAL EXECUTIONS-API CLASSIFICATION (this code is pragma:no-cover, so a careful read is
the ONLY check).
In GoogleCloudRunJobClient.execution_terminal_state, verify against the google-cloud-run run_v2
Execution proto:
(a) using `completion_time.seconds` truthiness to mean "terminal" is correct (unset Timestamp =>
seconds 0 =>
still running); (b) the cancelled_count/failed_count/succeeded_count precedence is right and
matches how a
cancelled or failed execution actually reports; (c) the non-resolvable-name and absent-SDK
guards return None
(best-effort) rather than raising; (d) the final `return "Failed"` fallback is safe. Flag any
way this could
misclassify a still-RUNNING execution as terminal (which would abort a healthy run) ? that is
the worst case.
```

2. assistant (2026-07-07T15:48:15.696Z)

```
I'll start by examining the commit and the relevant files.
```

3. user (2026-07-07T15:48:18.912Z)

```
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
---BRANCH---
fix/cloud-run-exec-status-fastfail
---DIFF STAT---
 backend/compute/__init__.py   |   8 +-
 backend/compute/base.py       |  14 +++
 backend/compute/cloud_run.py  | 115 ++++++++++++++++++++++-
 backend/worker/__main__.py    |  16 ++-
 tests/test_compute_backend.py | 229 ++++++++++++++++++++++++++++++++++++++++++-
 5 files changed, 375 insertions(+), 7 deletions(-)
```

4. assistant (2026-07-07T15:48:21.070Z)

```
I'll begin by reading the full changed files and the sibling context.
```

5. user (2026-07-07T15:48:23.508Z)

```
diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
index 1091582..482f3c9 100644
--- a/backend/compute/__init__.py
+++ b/backend/compute/__init__.py
@@ -13,13 +13,19 @@ import dataclasses
 import os
 from typing import Any, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,
+    RemoteExecutionFailed,
+)
 from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
```

```
 __all__ = [
     "ComputeBackend",
     "ComputeJobSpec",
     "ComputeResult",
+    "RemoteExecutionFailed",
     "get_compute_backend",
 ]

diff --git a/backend/compute/base.py b/backend/compute/base.py
index 31af3fe..3ad76fc 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -17,6 +17,20 @@ from dataclasses import dataclass, field
 from typing import Callable, Optional


+class RemoteExecutionFailed(RuntimeError):
+    """A dispatched remote job's execution reached a TERMINAL state without ever writing its
+    completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image argparse-
rejecting
+    an unknown flag) before the marker push, or the marker upload itself failed on an otherwise
+    Succeeded run.
+
+    Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL be
+    running): this says the execution is DEFINITIVELY over and no marker will ever appear, so
the
+    dispatcher must stop bucket-polling immediately instead of burning the full
+    ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message carries the
+    job + execution name + terminal state + the ``gcloud`` command to read the execution log,
so
+    the whole "worker crashed before the done-marker" class fails fast AND diagnostically."""
+
+
 @dataclass(frozen=True)
 class ComputeJobSpec:
     """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will (re)compute
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 7247ace..1e6e313 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -11,6 +11,10 @@ Flow (the cloud-serving compute path):
     job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-
healthy
     worker that finished just past the deadline is rescued, not wasted ? then best-effort
CANCELS
     the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
+    *Fast negative path:* while the marker stays the SOLE success signal, each poll also
probes the
+    Cloud Run Executions API ? if the execution has TERMINATED without ever writing a marker
(a
+    worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
+    (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile wait.
  3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
     ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

@@ -32,7 +36,12 @@ import uuid
 from abc import ABC, abstractmethod
 from typing import Any, Callable, List, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,
```

```
+    RemoteExecutionFailed,
+)
 from backend.infrastructure.execution_policy import (
     DEFAULT_POLICY,
     ExecutionPolicy,
@@ -76,6 +85,21 @@ class CloudRunJobClient(ABC):
         May raise ? the caller treats every failure as best-effort (the original poll timeout
is
         NEVER masked by a cancel error)."""

+    def execution_terminal_state(self, execution: str) -> Optional[str]:
+        """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
+        ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or the state
+        can't be determined).
+
+        ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a worker
that
+        terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails fast
instead
+        of bucket-polling the full budget. The done-marker stays the ONLY success signal.
+
+        NON-abstract with a default of ``None`` (additive / default-OFF): a client that can't ?
or
+        chooses not to ? poll the Executions API degrades transparently to today's marker-only
+        polling. Implementations MAY raise on a transient API error ? ``run_infer`` treats a
raise
+        as ``None`` (unknown ? keep waiting), so a flaky status API never fails a healthy
run."""
+        return None
+

 class GoogleCloudRunJobClient(CloudRunJobClient):
     """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI
uses
@@ -145,6 +169,35 @@ class GoogleCloudRunJobClient(CloudRunJobClient):
         )  # pragma: no cover - exercised only on a real run
         client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))

+    def execution_terminal_state(self, execution: str) -> Optional[str]:
+        # Advisory status probe (see the ABC docstring): describe the execution and, ONLY once
it
+        # is terminal (``completion_time`` set), classify it. A non-resolvable name / absent
SDK
+        # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
+        # raises on bad input, because the probe is best-effort and must not fail a healthy
run.
+        if "/executions/" not in (execution or ""):
+            return None
+        try:
+            from google.cloud import run_v2  # type: ignore[attr-defined]
+        except Exception:  # pragma: no cover - real SDK not present in CI
+            return None
+        client = (
+            run_v2.ExecutionsClient()
+        )  # pragma: no cover - exercised only on a real run
+        exe = client.get_execution(  # pragma: no cover - real run only
+            request=run_v2.GetExecutionRequest(name=execution)
+        )
+        # completion_time is a proto Timestamp: unset ? seconds==0 ? still running/pending.
+        if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
+            return None  # pragma: no cover - real run only
+        # Terminal: prefer the explicit per-task counts (a cancel shows as cancelled_count>0).
+        if getattr(exe, "cancelled_count", 0):  # pragma: no cover - real run only
+            return "Cancelled"
```

```
+        if getattr(exe, "failed_count", 0):  # pragma: no cover - real run only
+            return "Failed"
+        if getattr(exe, "succeeded_count", 0):  # pragma: no cover - real run only
+            return "Succeeded"
+        return "Failed"  # pragma: no cover - terminal but no positive success ? treat as
failed
+

 class CloudRunCompute(ComputeBackend):
     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
@@ -215,7 +268,7 @@ class CloudRunCompute(ComputeBackend):

         try:
             marker = poll_until(
-                lambda: self._read_marker(done_uri),
+                lambda: self._poll_check(done_uri, str(execution or "")),
                 self._policy,
                 label="cloud-run-infer",
                 logger=LOG,
@@ -224,6 +277,9 @@ class CloudRunCompute(ComputeBackend):
                 on_tick=on_wait,
             )
         except PolicyTimeout as timeout_exc:
+            # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
+            # so it propagates past here uncaught ? the dispatch handler maps it to a distinct,
+            # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
         video_id = str(marker.get("video_id", ""))
         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
@@ -249,6 +305,61 @@ class CloudRunCompute(ComputeBackend):
             execution=str(execution or ""),
         )

+    def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
+        """One bucket-poll iteration, hardened with an Executions-API fast negative path.
+
+        The done-marker is the SOLE success signal and is checked FIRST ? a present marker
always
+        wins (even if the execution's status later flips), so a worker that wrote a SUCCESS *or
a
+        FAILURE* marker resolves through the normal path with its real returncode. Only when
the
+        marker is absent do we consult the execution status: a worker that TERMINATED without
ever
+        writing a marker (crash / OOM / an old image argparse-aborting on an unknown flag)
would
+        otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min) before
failing
+        opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
+        failure.
+
+        Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
+        ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over, no
marker
+        will ever appear)."""
+        marker = self._read_marker(done_uri)
+        if marker is not None:
+            return marker  # success signal ? authoritative even if status later flips
+        state = self._terminal_execution_state(execution)
+        if state is None:
+            return None  # still pending/running, or status unknown ? keep bucket-polling
```

```
+        # The execution is terminal but no marker is present. Re-check the marker ONCE to close
the
+        # race where the worker wrote it between our two reads (mirrors the post-timeout rescue
in
+        # _handle_poll_timeout). If it is genuinely absent, the process is gone and no marker
will
+        # EVER appear ? fail fast + diagnostically rather than wait out the whole poll budget.
+        marker = self._read_marker(done_uri)
+        if marker is not None:
+            return marker
+        raise RemoteExecutionFailed(
+            f"cloud-run: job={self._job_name} execution={execution or '<unknown>'} reached "
+            f"terminal state {state!r} with NO completion marker ? the worker exited before "
+            "writing one (crash / OOM / pre-marker abort, or a failed marker upload on an "
+            "otherwise-Succeeded run). The execution is over, so the marker will never appear;
"
+            "inspect the Cloud Run Job execution log to diagnose: "
+            f"`gcloud run jobs executions describe {execution or '<execution>'} "
+            f"--region {self._region}`."
+        )
+
+    def _terminal_execution_state(self, execution: str) -> Optional[str]:
+        """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A raising
or
+        flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
+        so any exception ? and an empty/unresolvable execution name ? degrades to ``None``
(unknown
+        ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
+        if not execution:
+            return None
+        try:
+            return self._client.execution_terminal_state(execution)
+        except Exception:  # noqa: BLE001 ? advisory probe: never let it kill the bucket-poll
+            LOG.debug(
+                "cloud-run: execution status probe failed for %s ? treating as unknown",
+                execution,
+                exc_info=True,
+            )
+            return None
+
     def _handle_poll_timeout(
         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
     ) -> dict:
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 9b19215..6d3420b 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -176,9 +176,14 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
     container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.

-    A remote job that never writes a completion marker (a failed/hung container) makes the
bucket
-    poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote
dispatch
-    timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
+    A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
+    from the local 2/3), never a raw traceback:
+      * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution may
still be
```

```
+        running (the shim best-effort cancels it).
+      * rc 5 ? the execution has TERMINATED without a marker (``RemoteExecutionFailed``): a
worker
+        crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead of the
+        ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
+    from backend.compute import RemoteExecutionFailed
     from backend.infrastructure.execution_policy import PolicyTimeout

     spec = ComputeJobSpec(
@@ -190,6 +195,11 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
     )
     try:
         result = compute.run_infer(spec)
+    except RemoteExecutionFailed as e:
+        logger.error(
+            "[worker] remote execution terminated without a completion marker: %s", e
+        )
+        return 5
     except PolicyTimeout as e:
         logger.warning(
             "[worker] remote compute did not complete (no marker within budget): %s", e
diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py
index faa4981..07e8068 100644
--- a/tests/test_compute_backend.py
+++ b/tests/test_compute_backend.py
@@ -12,7 +12,11 @@ import tempfile

 import pytest

-from backend.compute import ComputeJobSpec, get_compute_backend
+from backend.compute import (
+    ComputeJobSpec,
+    RemoteExecutionFailed,
+    get_compute_backend,
+)
 from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
 from backend.config import AppConfig
 from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout
@@ -110,6 +114,37 @@ class _NeverDoneClient(CloudRunJobClient):
             raise self._cancel_error


+class _CrashedNoMarkerClient(CloudRunJobClient):
+    """Simulates the bug's failure class: the worker execution TERMINATES (crash / OOM / an old
+    image argparse-aborting on an unknown flag) WITHOUT ever writing a done-marker. Dispatch
returns
+    a resolvable execution name, no marker is ever written, and the Executions API reports a
terminal
+    state ? so run_infer must fail FAST via the status probe instead of spinning the poll
budget."""
+
+    EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-crash"
+
+    def __init__(self, *, state: str = "Failed"):
+        self.calls: list[dict] = []
+        self.cancelled: list[str] = []
+        self.status_polls = 0
+        self._state = state
+
+    def run_job(self, job_name, argv, *, region, project=None):
+        self.calls.append(
+            {"job": job_name, "argv": list(argv), "region": region, "project": project}
+        )
+        return self.EXEC_NAME
+
```

```
+    def cancel_execution(self, execution):
+        self.cancelled.append(execution)
+
+    def execution_terminal_state(self, execution):
+        self.status_polls += 1
+        # Anti-hang guard: the fast path MUST abort on the first terminal observation. If a
+        # regression kept polling despite a terminal status, trip here instead of spinning.
+        assert self.status_polls <= 5, "status probed repeatedly ? fast path did not abort the
poll"
+        return self._state
+
+
 # ------------------------------------------------------------------ factory (default-OFF)


@@ -423,6 +458,198 @@ def test_poll_timeout_late_marker_rescues_result(monkeypatch):
     assert exists_calls["n"] == 2  # exactly ONE extra post-timeout check


+# -------------------------------- execution-status fast negative path (worker crashed pre-
marker)
+
+# poll_every_n_sec=0.0 ? the poll spins as fast as possible; max_wait_sec=2.0 BOUNDS any
regression
+# (a fast path that silently fell back to marker-only polling fails in ~2s here, never a 65-min
hang)
+# while the working fast path aborts on poll 0 ? instantly.
+_FAILFAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=2.0)
+
+
+def test_run_infer_fails_fast_when_execution_terminal_without_marker(tmp_path):
+    """The core fix: a worker execution that TERMINATED (Failed) without ever writing a marker
is
+    caught by the Executions-API probe ? RemoteExecutionFailed FAST (poll 0), not a ~65-min
opaque
+    hang. The diagnostic names the execution, the terminal state, and the log-inspection
command."""
+    client = _CrashedNoMarkerClient(state="Failed")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )
+    with pytest.raises(RemoteExecutionFailed) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    msg = str(exc.value)
+    assert _CrashedNoMarkerClient.EXEC_NAME in msg  # operator can find the execution
+    assert "Failed" in msg  # the terminal state is surfaced
+    assert "gcloud run jobs executions describe" in msg  # actionable log pointer
+    assert client.status_polls == 1  # aborted on the FIRST terminal observation (fast, not
spun)
+    assert client.cancelled == []  # a finished execution is never cancelled (nothing to
cancel)
+
+
+def test_dispatch_remote_maps_terminal_without_marker_to_rc5(tmp_path):
+    """The dispatch handler maps RemoteExecutionFailed to a distinct, clearly-labelled rc 5 ?
NOT
+    the rc 4 poll-timeout path (a still-maybe-running execution) and NOT a raw traceback ? so
+    'worker crashed before the marker' is operationally distinguishable and surfaces
immediately."""
```

```
+    import argparse
+
+    from backend.worker import __main__ as wmain
+
+    client = _CrashedNoMarkerClient(state="Failed")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )
+    args = argparse.Namespace(
+        video_uri="gs://b/v.mp4",
+        out_uri=str(tmp_path / "bucket"),
+        segmenter="wasb_hybrid",
+        set=None,
+    )
+    rc = wmain._dispatch_remote(backend, args)
+    assert rc == 5  # the crashed-pre-marker fast-fail code...
+    assert rc != 4  # ...distinct from the poll-budget timeout
+    assert client.status_polls == 1  # reached the fast path (not the timeout branch)
+
+
+def test_succeeded_state_without_marker_also_fails_fast(tmp_path):
+    """Generalization: ANY terminal state with no marker is a guaranteed-futile wait. A
'Succeeded'
+    execution that nonetheless wrote no marker (the marker upload itself failed) also fails
fast,
+    with the state surfaced ? the whole 'execution over, no marker' class collapses, not just
Failed."""
+    client = _CrashedNoMarkerClient(state="Succeeded")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )
+    with pytest.raises(RemoteExecutionFailed) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert "Succeeded" in str(exc.value)
+
+
+def test_present_marker_wins_over_terminal_status(tmp_path):
+    """The done-marker stays AUTHORITATIVE: a worker that wrote a marker ? even a FAILURE
marker
+    (rc 2) ? resolves through the normal path with its REAL returncode, never a synthesized
+    RemoteExecutionFailed, even when the execution status also reads terminal. Status is a
negative
+    path only; it must never override a real marker (which is checked first)."""
+
+    class _MarkerAndFailedStatus(_FakeRunClient):
+        def execution_terminal_state(self, execution):  # pragma: no cover - never reached
+            return "Failed"  # the marker resolves poll 0, so this is never consulted
+
+    client = _MarkerAndFailedStatus(video_id="abc", returncode=2)
+    backend = CloudRunCompute(
+        run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
+    )
+    result = backend.run_infer(
+        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+    )
```

```
+    assert result.returncode == 2  # the worker's REAL rc, not RemoteExecutionFailed
+    assert result.video_id == "abc"
+
+
+def test_status_probe_error_is_swallowed_and_falls_back_to_marker_polling(tmp_path):
+    """Best-effort probe: a raising Executions API must NEVER fail a healthy run. The wait
degrades
+    to today's marker-only polling ? PolicyTimeout (the existing timeout path), NOT
+    RemoteExecutionFailed ? a flaky status API cannot manufacture a failure."""
+
+    class _RaisingStatusClient(_NeverDoneClient):
+        def execution_terminal_state(self, execution):
+            raise RuntimeError("executions API unreachable")
+
+    client = _RaisingStatusClient()
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_TIMEOUT_NOW,
+        token="tok",
+    )
+    with pytest.raises(PolicyTimeout):  # swallowed probe error ? the unchanged timeout path
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # still cancels on the real
timeout
+
+
+def test_default_execution_terminal_state_is_none():
+    """The ABC default is None (additive / default-OFF): a client that doesn't override status
+    polling degrades transparently to marker-only behaviour, so every pre-existing fake is
+    unaffected and the seam never changes a healthy run's outcome."""
+    assert _NoopRunClient().execution_terminal_state("whatever") is None
+    assert (
+        _NeverDoneClient().execution_terminal_state(_NeverDoneClient.EXEC_NAME) is None
+    )
+
+
+def test_real_client_terminal_state_none_for_unresolvable_name():
+    """The real client's status probe is best-effort: a non-resolvable execution name returns
None
+    (can't probe ? keep marker-polling), NOT a raise ? exercisable in CI (the SDK is
absent)."""
+    from backend.compute.cloud_run import GoogleCloudRunJobClient
+
+    client = GoogleCloudRunJobClient()
+    for bogus in ("", "execution", "exec-123"):
+        assert client.execution_terminal_state(bogus) is None
+
+
+def test_terminal_status_with_marker_written_in_race_resolves_via_marker(monkeypatch):
+    """Concurrency race: the execution reads terminal exactly as the worker finishes writing
its
+    marker. _poll_check re-checks the marker ONCE after a terminal status ? if it landed in
between,
+    the completed result is ingested (the marker wins), NOT a spurious
RemoteExecutionFailed."""
+    from backend.compute import cloud_run as cr
+
+    exists_calls = {"n": 0}
+
+    class _FakeStore:
+        def exists(self, ref):
```

```
+            exists_calls["n"] += 1
+            # absent on the pre-status read, present on the ONE post-status race re-check
+            return exists_calls["n"] >= 2
+
+    monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
+
+    def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+        if dst is None:
+            dst = os.path.join(tempfile.mkdtemp(prefix="cr-race-"), "m.json")
+        with open(dst, "w", encoding="utf-8") as f:
+            f.write('{"video_id": "vrace", "returncode": 0, "segment_count": 1}')
+        return dst
+
+    monkeypatch.setattr(cr, "fetch", fake_fetch)
+    client = _CrashedNoMarkerClient(state="Failed")  # status says terminal-Failed...
+    backend = cr.CloudRunCompute(
+        run_client=client, job_name="j", region="r", token="t", policy=_FAILFAST
+    )
+    result = backend.run_infer(
+        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
+    )
+    assert result.video_id == "vrace"  # ...but the marker that raced in WINS
+    assert exists_calls["n"] == 2  # exactly the pre-status read + the ONE race re-check
+    assert client.status_polls == 1  # status was consulted once, then the marker superseded it
+
+
+def test_terminal_execution_state_none_for_empty_execution():
+    """An empty/unknown execution name (run_job could not resolve one) can't be probed ? None
+    (degrade to marker-only polling), and the client's status API is never even called."""
+
+    class _Boom(_NoopRunClient):
+        def execution_terminal_state(self, execution):  # pragma: no cover - must never be
called
+            raise AssertionError("must not probe an empty execution name")
+
+    backend = CloudRunCompute(run_client=_Boom(), job_name="j", region="r")
+    assert backend._terminal_execution_state("") is None
+
+
 # ---------------------------------------------------- worker wiring: dispatch branch + done-
marker
```

6. assistant (2026-07-07T15:48:25.876Z)

Now let me read the full current files to judge correctness in context.

7. user (2026-07-07T15:48:27.276Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
```

```
          job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
          worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
          the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
          *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
          Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
          worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
          (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
       3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
          ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.

    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
    owner-verified on the M7 real run.
    """

    from __future__ import annotations

    import json
    import logging
    import os
    import tempfile
    import uuid
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional

    from backend.compute.base import (
        ComputeBackend,
        ComputeJobSpec,
        ComputeResult,
        RemoteExecutionFailed,
    )
    from backend.infrastructure.execution_policy import (
        DEFAULT_POLICY,
        ExecutionPolicy,
        PolicyTimeout,
        poll_until,
    )
    from backend.storage import fetch, get_storage, parse_uri

    LOG = logging.getLogger("compute.cloud_run")

    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


```

```
63      class CloudRunJobClient(ABC):
64          """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
65
66          Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
67          overrides the job's container ``args`` for this execution and starts it."""
68
69          @abstractmethod
70          def run_job(
71              self,
72              job_name: str,
73              argv: List[str],
74              *,
75              region: str,
76              project: Optional[str] = None,
77          ) -> str:
78              """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
79              execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
80
81          @abstractmethod
82          def cancel_execution(self, execution: str) -> None:
83              """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
84              times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
85              May raise ? the caller treats every failure as best-effort (the original poll
timeout is
86              NEVER masked by a cancel error)."""
87
88          def execution_terminal_state(self, execution: str) -> Optional[str]:
89              """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
90              ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
91              can't be determined).
92
93              ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
94              terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95              of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97              NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98              chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99              polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100             as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101             return None
102
103
104     class GoogleCloudRunJobClient(CloudRunJobClient):
105         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108         def run_job(
109             self,
```

```
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
            # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
            # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
            if not project:
                raise RuntimeError(
                    "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
                    "to resolve the job path; set it on the control plane (see
get_compute_backend)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.JobsClient()
            )  # pragma: no cover - exercised only on the real M7 run
            name = client.job_path(project, region, job_name)
            overrides = run_v2.RunJobRequest.Overrides(
                container_overrides=[
                    run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
                ]
            )
            op = client.run_job(
                request=run_v2.RunJobRequest(name=name, overrides=overrides)
            )
            return (
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
            )

        def cancel_execution(self, execution: str) -> None:
            # run_job falls back to the literal string "execution" when the operation
metadata carries
            # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
            # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
            # google-cloud-run is absent).
            if "/executions/" not in (execution or ""):
                raise RuntimeError(
                    f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
                    "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
                    "one from the operation metadata, so there is nothing to cancel by
name)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
```

```
                raise RuntimeError(
                    "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))

        def execution_terminal_state(self, execution: str) -> Optional[str]:
            # Advisory status probe (see the ABC docstring): describe the execution and,
ONLY once it
            # is terminal (``completion_time`` set), classify it. A non-resolvable name /
absent SDK
            # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this
never
            # raises on bad input, because the probe is best-effort and must not fail a
healthy run.
            if "/executions/" not in (execution or ""):
                return None
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception:  # pragma: no cover - real SDK not present in CI
                return None
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            exe = client.get_execution(  # pragma: no cover - real run only
                request=run_v2.GetExecutionRequest(name=execution)
            )
            # completion_time is a proto Timestamp: unset ? seconds==0 ? still
running/pending.
            if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
                return None  # pragma: no cover - real run only
            # Terminal: prefer the explicit per-task counts (a cancel shows as
cancelled_count>0).
            if getattr(exe, "cancelled_count", 0):  # pragma: no cover - real run only
                return "Cancelled"
            if getattr(exe, "failed_count", 0):  # pragma: no cover - real run only
                return "Failed"
            if getattr(exe, "succeeded_count", 0):  # pragma: no cover - real run only
                return "Succeeded"
            return "Failed"  # pragma: no cover - terminal but no positive success ? treat
as failed


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(
            self,
            *,
            run_client: CloudRunJobClient,
            job_name: str,
            region: str,
            project: Optional[str] = None,
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
```

```
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
                if container_overrides is None
                else tuple(container_overrides)
            )

        def run_infer(
            self,
            spec: ComputeJobSpec,
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            if spec.skip_validation:
                # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
                # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
                argv.append("--skip-validation")
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            try:
                marker = poll_until(
                    lambda: self._poll_check(done_uri, str(execution or "")),
                    self._policy,
                    label="cloud-run-infer",
                    logger=LOG,
                    # Live heartbeat (#482): each still-waiting tick reaches the caller so
                    # job.progress moves during the whole GPU run instead of freezing.
                    on_tick=on_wait,
                )
            except PolicyTimeout as timeout_exc:
                # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
```

```
PolicyTimeout,
281                 # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
282                 # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
283                 marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
284             video_id = str(marker.get("video_id", ""))
285             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
286             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
287             rc = int(marker.get("returncode", 1))
288             if not video_id:
289                 LOG.warning(
290                     "cloud-run: job=%s completion marker present but empty/unreadable (no "
291                     "video_id) ? treating as failure",
292                     self._job_name,
293                 )
294                 rc = rc or 1
295             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
296             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
297             LOG.info(
298                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
299             )
300             return ComputeResult(
301                 returncode=rc,
302                 outputs_uri=outputs_uri,
303                 video_id=video_id,
304                 report=report,
305                 execution=str(execution or ""),
306             )
307
308         def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
309             """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
310
311             The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
312             wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
313             FAILURE* marker resolves through the normal path with its real returncode. Only
when the
314             marker is absent do we consult the execution status: a worker that TERMINATED
without ever
315             writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
316             otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
317             opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
318             failure.
319
320             Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
321             ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
322             will ever appear)."""
323             marker = self._read_marker(done_uri)
324             if marker is not None:
325                 return marker  # success signal ? authoritative even if status later flips
326             state = self._terminal_execution_state(execution)
327             if state is None:
328                 return None  # still pending/running, or status unknown ? keep bucket-
polling
329             # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
```

```
330             # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
331             # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
332             # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
333             marker = self._read_marker(done_uri)
334             if marker is not None:
335                 return marker
336             raise RemoteExecutionFailed(
337                 f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
338                 f"terminal state {state!r} with NO completion marker ? the worker exited
before "
339                 "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
340                 "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
341                 "inspect the Cloud Run Job execution log to diagnose: "
342                 f"`gcloud run jobs executions describe {execution or '<execution>'} "
343                 f"--region {self._region}`."
344             )
345
346         def _terminal_execution_state(self, execution: str) -> Optional[str]:
347             """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
348             flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
349             so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
350             ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
351             if not execution:
352                 return None
353             try:
354                 return self._client.execution_terminal_state(execution)
355             except Exception:  # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
356                 LOG.debug(
357                     "cloud-run: execution status probe failed for %s ? treating as unknown",
358                     execution,
359                     exc_info=True,
360                 )
361                 return None
362
363         def _handle_poll_timeout(
364             self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
365         ) -> dict:
366             """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
367
368             The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
369             ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
370             ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
371             is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
372             L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
373             execution name so the operator can verify the state in the GCP console. A cancel
failure
374             NEVER masks the original timeout."""
375             try:
376                 late = self._read_marker(done_uri)
377             except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
```

```
? failure
                LOG.warning(
                    "cloud-run: final post-timeout marker check failed ? treating as
absent",
                    exc_info=True,
                )
                late = None
            if late is not None:
                LOG.warning(
                    "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
                    "? rescuing the completed result",
                    self._job_name,
                    execution,
                )
                return late
            try:
                self._client.cancel_execution(execution)
            except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
                cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
                LOG.warning(
                    "cloud-run: best-effort cancel of execution=%s failed: %s",
                    execution,
                    cancel_exc,
                    exc_info=True,
                )
            else:
                cancel_note = "cancel requested"
                LOG.info("cloud-run: requested cancel of execution=%s", execution)
            raise PolicyTimeout(
                f"{timeout_exc} [cloud-run job={self._job_name} "
                f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
                "state in the GCP console]"
            ) from timeout_exc

        # --- storage-backed completion signal (bucket-poll) ---
        def _read_marker(self, done_uri: str) -> Optional[dict]:
            """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
            ref = parse_uri(done_uri)
            if not get_storage(ref.backend, config=self._storage_config).exists(ref):
                return None
            return (
                self._read_json(done_uri) or {}
            )  # empty dict still terminates the poll (present == done)

        def _read_json(self, uri: str) -> dict:
            # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
            # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
            # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
            try:
                ref = parse_uri(uri)
                with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
                    dst = os.path.join(td, ref.name or "obj.json")
                    local = fetch(uri, dst, config=self._storage_config)
                    with open(local, "r", encoding="utf-8") as f:
                        data: Any = json.load(f)
                return data if isinstance(data, dict) else {}
            except Exception:
                LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
```

```
434                 return {}
435
```

8. assistant (2026-07-07T15:48:30.111Z)

Now let me read the other files and sibling context.

9. user (2026-07-07T15:48:31.867Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11      """
12
13      from __future__ import annotations
14
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
17      from typing import Callable, Optional
18
19
20      class RemoteExecutionFailed(RuntimeError):
21          """A dispatched remote job's execution reached a TERMINAL state without ever writing
its
22          completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image
argparse-rejecting
23          an unknown flag) before the marker push, or the marker upload itself failed on an
otherwise
24          Succeeded run.
25
26          Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL
be
27          running): this says the execution is DEFINITIVELY over and no marker will ever
appear, so the
28          dispatcher must stop bucket-polling immediately instead of burning the full
29          ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message
carries the
30          job + execution name + terminal state + the ``gcloud`` command to read the execution
log, so
31          the whole "worker crashed before the done-marker" class fails fast AND
diagnostically."""
32
33
34      @dataclass(frozen=True)
35      class ComputeJobSpec:
36          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
37          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
38          deterministic regardless of where the job ran."""
39
40          video_uri: str  # input (gs://? for cloud; a path/local:// for local)
41          out_uri: str  # output ROOT (gs://? bucket for cloud)
42          segmenter: Optional[str] = None
43          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
```

```
frozen/hashable.
44          config_overrides: tuple[str, ...] = ()
45          # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving
control-plane) is
46          # the authoritative validation gate: it runs the SAME validators with ITS resolved
config and only
47          # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video
decode, and
48          # (c) outright WRONG when the worker's config resolves different thresholds ? it can
REJECT a clip
49          # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no
control-plane)
50          # validating exactly as before.
51          skip_validation: bool = False
52
53
54      @dataclass(frozen=True)
55      class ComputeResult:
56          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
57          (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
58          empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
59          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
60
61          returncode: int
62          outputs_uri: str = ""
63          video_id: str = ""
64          report: dict = field(default_factory=dict)
65          # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
66          # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
67          # executions describe <execution>). Empty for the in-process path / backends without
one.
68          execution: str = ""
69
70
71      class ComputeBackend(ABC):
72          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
73          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
74          execution identically."""
75
76          @abstractmethod
77          def run_infer(
78              self,
79              spec: ComputeJobSpec,
80              *,
81              on_wait: "Callable[[float], None] | None" = None,
82          ) -> ComputeResult:
83              """Run ``spec`` to completion and return its ``ComputeResult``.
84
85              ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
86              still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
87              long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
88
```

10. user (2026-07-07T15:48:40.676Z)

```
"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.

    python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
    python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)

The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://``
is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs
are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

This is a thin orchestrator: it composes the SAME building blocks the ``backend.main`` CLI uses
(validator registry ? segmenter ? report), never a forked pipeline.
"""

from __future__ import annotations

import argparse
import csv
import json
import logging
import os
import re
import sys
import tempfile
from typing import Any, List

from backend import storage
from backend.compute import ComputeJobSpec, get_compute_backend
from backend.config import (
    StartupCheckError,
    enforce_startup_checks,
    load_config,
    probe_cuda_available,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.validators.base import registry as validator_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
from backend.utils.hashing import compute_video_id

logger = logging.getLogger("backend.worker")


def _coerce(v: str) -> Any:
    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
    low = v.lower()
    if low in ("true", "false"):
        return low == "true"
    for cast in (int, float):
        try:
            return cast(v)
        except ValueError:
            pass
    return v


```

```
    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _serialize_and_push_outputs(
        scratch: str,
        out_uri: str,
        video_id: str,
        segments: List[dict],
        status: str,
        storage_cfg: dict,
```

```
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
134         out_local = os.path.join(scratch, "outputs", video_id)
135         os.makedirs(out_local, exist_ok=True)
136         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137         _write_report_json(
138             video_id,
139             segments,
140             status,
141             os.path.join(out_local, "report.json"),
142             video_uri=video_uri,
143         )
144         out_prefix = out_uri.rstrip("/")
145         pushed: List[str] = []
146         for fn in sorted(os.listdir(out_local)):
147             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False  # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
```

```
        container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.

        A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
        from the local 2/3), never a raw traceback:
          * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
            running (the shim best-effort cancels it).
          * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
            crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
            ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
        from backend.compute import RemoteExecutionFailed
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
            video_uri=args.video_uri,
            out_uri=args.out_uri,
            segmenter=args.segmenter,
            config_overrides=tuple(args.set or ()),
            skip_validation=bool(getattr(args, "skip_validation", False)),
        )
        try:
            result = compute.run_infer(spec)
        except RemoteExecutionFailed as e:
            logger.error(
                "[worker] remote execution terminated without a completion marker: %s", e
            )
            return 5
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
            result.video_id,
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
```

```
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
        # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
        # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
        # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
        # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
        # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
        # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
        # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
        # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
        if getattr(args, "skip_validation", False):
            print(
                "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
```

```
                "gate and already validated this input."
            )
            val_results, val_context = [], {}
        else:
            val_results, val_context = validator_registry.run_all_with_context(
                local_video, config
            )
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            [r.model_dump() for r in val_results],
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
```

```
343                     f"[worker] unknown segmenter: {seg_name!r} "
344                     f"(have {list(segmenter_registry.list_available())})"
345                 )
346                 return _fail(2)
347             segmenter_actual = seg_name
348             result = segmenter_cls(db, config).process_video(
349                 video_id, local_video, max_frames=args.max_frames
350             )
351             segmentation_ran = True
352             if isinstance(result, Failure):
353                 print(f"[worker] segmenter FAILED: {result.message}")
354                 return _fail(3)
355             segments = result.value if hasattr(result, "value") else result
356             status = "success"
357             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
360             if not segments:
361                 detector_impl = (
362                     os.environ.get("RALLY_DETECTOR_IMPL")
363                     or getattr(config.indexing, "detector_impl", None)
364                     or "auto"
365                 )
366                 logger.warning(
367                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
368                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369                     "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
370                     "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
371                     "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
372                     "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
373                     "detections, or the input resolution differs from the tuned proxy
resolution. "
374                     "Re-run with -v for the native command + frame counts.",
375                     video_id,
376                     segmenter_actual,
377                     detector_impl,
378                 )
379         finally:
380             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
381             emit_indexer_report(
382                 db=db,
383                 video_id=video_id,
384                 video_path=local_video,
385                 config=config,
386                 val_results=val_results,
387                 val_context=val_context,
388                 segmenter_requested=args.segmenter,
389                 segmenter_actual=segmenter_actual,
390                 was_reingest=False,
391                 segmentation_ran=segmentation_ran,
392             )
393
394         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395         pushed = _serialize_and_push_outputs(
396             scratch,
397             args.out_uri,
```

```
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
        )

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

    #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")


    def _label_clip_name(fname: str) -> str:
        """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.

        Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
        canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

        * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
        * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
        * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
        * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)

        Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
        video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
        """
```

```
        name = fname.replace(".rallies.csv", "").replace(".csv", "")
        return _LABEL_DATE_PREFIX.sub("", name)


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

        The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
        fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
        cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
        inference report (it flags the fallback) but corrupting for REAL training data. So
here we
        probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
        rather than guesses.
        """
        import json
        import subprocess

        try:
            out = subprocess.check_output(
                [
                    ffprobe_bin(),
                    "-v",
                    "error",
                    "-select_streams",
                    "v:0",
                    "-show_entries",
                    "stream=r_frame_rate,width",
                    "-of",
                    "json",
                    path,
                ],
                stderr=subprocess.DEVNULL,
            ).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
```

```
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

```

```
        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(
            u
            for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
            if u.lower().endswith(".csv")
        )
        if len(label_uris) < 2:
            print(
                f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                f"under {corpus}/labels/"
            )
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
            name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
            local_label = storage.fetch(
                uri, os.path.join(labels_dir, fname), config=storage_cfg
            )
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(
                        f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
                    )
                    continue
                local_video = storage.fetch(
                    vuri,
                    os.path.join(videos_dir, storage.parse_uri(vuri).name),
                    config=storage_cfg,
                )
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
```

```
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
```

```
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
            "--verbose",
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
```

```
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.add_argument(
            "--skip-validation",
            action="store_true",
            help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
            "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
            "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
            "`worker infer` CLI still validates.",
        )
        pi.set_defaults(func=cmd_infer)

        pt = sub.add_parser(
            "train",
            help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
        )
        pt.add_argument(
            "--corpus-uri",
            required=True,
            help="prefix containing labels/ (+ videos/ for "
            "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
        )
        pt.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
        pt.add_argument(
            "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
        )
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument(
            "--fps",
            type=float,
            default=30.0,
            help="synth trajectory fps (detector probes the video)",
        )
        pt.add_argument(
            "--frame-width",
            type=float,
            default=1280.0,
            help="synth trajectory frame width (detector probes the video)",
        )
        pt.add_argument("--set", action="append", default=[], metavar="k=v")
```

```
810         pt.set_defaults(func=cmd_train)
811         return p
812
813
814     def _setup_logging(verbose: bool) -> None:
815         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
816         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
817         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
818         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
819         entry), not at import, so importing the module never reconfigures a caller's
logging."""
820         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
821         # configured logging at import), which would leave the level at WARNING and re-
suppress the
822         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
823         logging.basicConfig(
824             level=logging.DEBUG if verbose else logging.INFO,
825             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
826             datefmt="%H:%M:%S",
827             force=True,
828         )
829
830
831     def main(argv=None) -> int:
832         args = build_parser().parse_args(argv)
833         _setup_logging(getattr(args, "verbose", False))
834         return args.func(args)
835
836
837     if __name__ == "__main__":
838         sys.exit(main())
839
```

11. assistant (2026-07-07T15:48:43.667Z)

Now let me read the remaining files: `__init__.py`, orchestration.py, execution_policy.py, and the sibling context diff.

12. user (2026-07-07T15:48:44.685Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import dataclasses
13      import os
14      from typing import Any, Optional
15
```

```
    from backend.compute.base import (
        ComputeBackend,
        ComputeJobSpec,
        ComputeResult,
        RemoteExecutionFailed,
    )
    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

    __all__ = [
        "ComputeBackend",
        "ComputeJobSpec",
        "ComputeResult",
        "RemoteExecutionFailed",
        "get_compute_backend",
    ]


    def _storage_dict(config: Any) -> dict:
        storage = getattr(config, "storage", None)
        if storage is None:
            return {}
        if hasattr(storage, "model_dump"):
            return storage.model_dump()  # type: ignore[no-any-return]
        return dict(storage) if isinstance(storage, dict) else {}


    def get_compute_backend(
        config: Any,
        *,
        run_client: Optional[Any] = None,
        policy: ExecutionPolicy = DEFAULT_POLICY,
        token: Optional[str] = None,
        target_override: Optional[str] = None,
    ) -> Optional[ComputeBackend]:
        """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
        in-process (the default-OFF path).

        Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
        Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
        / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
        pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).

        ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
        per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
        server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
        """
        target = (
            target_override
            if target_override is not None
            else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
        )
        if target != "cloud_run":
            return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

        # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
        from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
```

```
72
73          # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
74          # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
75          # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild §4.3; the
old
76          # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
77          # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
78          if policy is DEFAULT_POLICY:
79              max_wait = getattr(
80                  getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
81              )
82              if max_wait:
83                  policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
84
85          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
86          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
87          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
88          client = run_client if run_client is not None else GoogleCloudRunJobClient()
89          return CloudRunCompute(
90              run_client=client,
91              job_name=job_name,
92              region=region,
93              project=project,
94              storage_config=_storage_dict(config),
95              policy=policy,
96              token=token,
97          )
98
```

13. assistant (2026-07-07T15:48:47.126Z)

Now let me read the execution_policy.py to understand poll_until semantics, and the sibling context diff.

14. user (2026-07-07T15:48:47.988Z)

```
1       """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3       Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4       - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7       - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10      **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11      waiting (a fake clock advances instantly).
12
13      **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14      transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15      ``execution.policy`` logger to watch the poll trail.
16      """
17
18      from __future__ import annotations
19
20      import logging
```

```
    import threading
    import time as _time
    from dataclasses import asdict, dataclass
    from enum import Enum
    from typing import Callable, Optional, TypeVar

    T = TypeVar("T")

    #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
    LOG = logging.getLogger("execution.policy")


    class WaitMode(str, Enum):
        """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""

        WAIT_AND_RESUME = "wait_and_resume"  # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
        ABORT = "abort"  # one attempt; on hang/fail, stop immediately
        BLOCK = "block"  # bounded in-process blocking wait (no reschedule)


    class PolicyTimeout(TimeoutError):
        """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""


    @dataclass(frozen=True)
    class ExecutionPolicy:
        """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""

        mode: WaitMode = WaitMode.WAIT_AND_RESUME
        poll_every_n_sec: float = 10.0
        op_timeout_sec: float = (
            120.0  # HARD per-attempt deadline ? the hang-killer that was missing
        )
        max_wait_sec: float = 1800.0  # total wall budget for a poll loop
        max_retries: int = 5  # retries (=> max_retries+1 attempts) for failures/hangs
        backoff_base_sec: float = 3.0  # exponential: backoff_base * 2**attempt
        on_timeout: str = "reschedule"  # reschedule | partial | fail  (consumed by P2/P3)
        resumable: bool = True

        def to_dict(self) -> dict:
            d = asdict(self)
            d["mode"] = self.mode.value
            return d

        @classmethod
        def from_dict(cls, d: dict) -> "ExecutionPolicy":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            kw = {k: v for k, v in d.items() if k in known}
            if "mode" in kw and not isinstance(kw["mode"], WaitMode):
                kw["mode"] = WaitMode(kw["mode"])
            return cls(**kw)


    #: The sane default (also what a job gets if it declares no policy).
    DEFAULT_POLICY = ExecutionPolicy()


    def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
        """Deterministic exponential backoff (no jitter ? reproducible tests)."""
        return policy.backoff_base_sec * (2**attempt)
```

```


    def run_bounded(
        fn: Callable[[], T],
        policy: ExecutionPolicy = DEFAULT_POLICY,
        *,
        label: str = "op",
        logger: Optional[logging.Logger] = None,
        sleep: Callable[[float], None] = _time.sleep,
    ) -> T:
        """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with backoff.

        Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt hung, or re-raises the
        last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no retries).
        """
        log = logger or LOG
        attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
        last_exc: Optional[BaseException] = None
        for attempt in range(attempts):
            box: dict = {}
            done = threading.Event()

            # box/done bound as defaults (not closed over) ? the thread is started + joined within this
            # same iteration, but explicit binding keeps each attempt isolated and silences B023.
            def _runner(_box: dict = box, _done: threading.Event = done) -> None:
                try:
                    _box["val"] = fn()
                except (
                    BaseException
                ) as e:  # relay ANY error (incl. timeouts) to the caller thread
                    _box["exc"] = e
                finally:
                    _done.set()

            threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
            if done.wait(timeout=policy.op_timeout_sec):
                if "exc" in box:
                    last_exc = box["exc"]
                    log.info(
                        "%s: attempt %d/%d failed: %s",
                        label,
                        attempt + 1,
                        attempts,
                        last_exc,
                    )
                else:
                    if attempt:
                        log.info(
                            "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
                        )
                    return box["val"]  # type: ignore[return-value]
            else:
                last_exc = PolicyTimeout(
                    f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
                )
                log.warning(
                    "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
                    label,
                    attempt + 1,
                    attempts,
```

```
                    policy.op_timeout_sec,
                )
            if attempt < attempts - 1:
                wait = _backoff_sec(policy, attempt)
                log.debug("%s: backing off %.1fs before retry", label, wait)
                sleep(wait)
        assert last_exc is not None
        log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
        raise last_exc


    def poll_until(
        check: Callable[[], Optional[T]],
        policy: ExecutionPolicy = DEFAULT_POLICY,
        *,
        label: str = "op",
        logger: Optional[logging.Logger] = None,
        sleep: Callable[[float], None] = _time.sleep,
        monotonic: Callable[[], float] = _time.monotonic,
        on_tick: Optional[Callable[[float], None]] = None,
    ) -> T:
        """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
        ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.

        Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
        ``on_tick(elapsed_sec)`` (optional) fires once per still-waiting iteration ? the
caller's live
        progress heartbeat (#482). Best-effort by contract: a raising callback is swallowed
        (DEBUG-logged), never allowed to kill the wait it reports on.
        """
        log = logger or LOG
        start = monotonic()
        polls = 0
        log.info(
            "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
            label,
            policy.poll_every_n_sec,
            policy.max_wait_sec,
        )
        while True:
            result = check()
            if result is not None:
                log.info(
                    "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
                )
                return result
            elapsed = monotonic() - start
            if elapsed >= policy.max_wait_sec:
                log.warning(
                    "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
                )
                raise PolicyTimeout(
                    f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
                )
            polls += 1
            log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
            if on_tick is not None:
                try:
                    on_tick(elapsed)
                except Exception:  # noqa: BLE001 ? progress reporting must never kill the
wait
                    log.debug("%s: on_tick callback failed", label, exc_info=True)
```

```
201             sleep(policy.poll_every_n_sec)
202
```

15. assistant (2026-07-07T15:48:52.880Z)

Now the sibling context diff and orchestration caller.

16. user (2026-07-07T15:48:55.446Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
plane) is
+    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
and only
+    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
and
+    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
a clip
+    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
plane)
+    # validating exactly as before.
+    skip_validation: bool = False


 @dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
         ]
         if spec.segmenter:
             argv += ["--segmenter", spec.segmenter]
+        if spec.skip_validation:
+            # The control-plane already validated + only dispatches on PASS ? tell the worker
not to
+            # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
+            argv.append("--skip-validation")
         for kv in (*spec.config_overrides, *self._container_overrides):
             argv += ["--set", kv]

diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
         out_uri=args.out_uri,
         segmenter=args.segmenter,
         config_overrides=tuple(args.set or ()),
+        skip_validation=bool(getattr(args, "skip_validation", False)),
     )
     try:
         result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
```

```
     video_id = compute_video_id(local_video)

-    # 3. VALIDATE (same registry as the main CLI).
-    val_results, val_context = validator_registry.run_all_with_context(
-        local_video, config
-    )
+    # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+    # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+    # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+    # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+    # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+    # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+    # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+    # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+    # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+    if getattr(args, "skip_validation", False):
+        print(
+            "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+            "gate and already validated this input."
+        )
+        val_results, val_context = [], {}
+    else:
+        val_results, val_context = validator_registry.run_all_with_context(
+            local_video, config
+        )
     failures = [r for r in val_results if not r.passed]
     db = Database(os.path.join(scratch, config.output.db_name))
     db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
         metavar="k=v",
         help="config override, e.g. --set indexing.skip_ai_handoff=true",
     )
+    pi.add_argument(
+        "--skip-validation",
+        action="store_true",
+        help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+        "validators with its resolved config and only dispatches on PASS, so re-validating here
is "
+        "redundant, decodes the video again, and can contradict the gate. Default-OFF: the
direct "
+        "`worker infer` CLI still validates.",
+    )
     pi.set_defaults(func=cmd_infer)

     pt = sub.add_parser(
```

17. assistant (2026-07-07T15:48:59.345Z)

Let me examine the orchestration caller and how `run_infer` / rc codes are consumed at the control-plane level.

18. user (2026-07-07T15:49:01.569Z)

```
4:Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` / ``_ingest_remote_segments``),
391:def _maybe_dispatch_remote(
589:        result = compute.run_infer(
728:    # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
```

```
805:        remote = _maybe_dispatch_remote(
```

19. user (2026-07-07T15:49:05.900Z)

```
560         wasb_cfg = getattr(idx_cfg, "wasb", None)
561         if wasb_cfg is not None:
562             decode_backend = getattr(wasb_cfg, "decode_backend", None)
563             if decode_backend:
564                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565             batch_size = getattr(wasb_cfg, "batch_size", None)
566             if batch_size:
567                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
568             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569             if prefetch:
570                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
572         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
573         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
574         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
575         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
576         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
577         spec = ComputeJobSpec(
578             video_uri=req.video_path,
579             out_uri=out_root,
580             segmenter=seg_name,
581             config_overrides=tuple(overrides),
582             skip_validation=True,
583         )
584         try:
585             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
586             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
587             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
588             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
589             result = compute.run_infer(
590                 spec,
591                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
592             )
593         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
594             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
595             return _failed(f"cloud dispatch error: {e}", True)
596         if result.returncode != 0:
597             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
598         if result.video_id and result.video_id != video_id:
599             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
600             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
601             logger.warning(
602                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
603                 "and the worker hash; ingesting under the local id.",
604                 result.video_id,
605                 video_id,
606             )
607
608         notify("ingesting (cloud_run)")
609         storage_dict = (
```

```
            storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
            if hasattr(storage_cfg, "model_dump")
            else {}
        )
        try:
            _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
        except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
            logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
            return _failed(
                f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
                True,
            )

        # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
        segments = [
            s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
        ]
        _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
        # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
        # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
        provenance = {
            "path": "cloud_run",
            "requested": req.compute,
            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            "execution": result.execution,
            "outputs_uri": result.outputs_uri,
        }
        logger.info(
            "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
            result.execution,
            provenance["job"],
            provenance["region"],
            result.outputs_uri,
            video_id,
        )
        return (
            {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
                "compute": provenance,
            },
            True,
        )


    def _execute_processing(
        config=None, db=None, req=None, progress=None
    ) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
        module-level ``global_config``/``db_instance`` (backward-compatible with existing
        tests that pass only ``req``). Route handlers always pass explicit params; older
        test code that calls ``_execute_processing(req)`` still works.
```

```
668
669         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
670         Raises on unexpected internal errors ? the caller records those as a job failure
671         """
672         import backend.main as _main  # call-time seam: main's globals/model stay patchable
673
674         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
675         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
676         if config is not None and not isinstance(config, AppConfig):
677             # Old signature: _execute_processing(req, progress=None)
678             if req is None and db is not None and isinstance(db, _main.ProcessRequest):
679                 req = db
680                 db = None
681             elif req is None and isinstance(config, _main.ProcessRequest):
682                 req = config
683                 config = None
684         # Resolve config/db from module globals when not passed explicitly
685         if config is None:
686             config = _main.global_config
687         if db is None:
688             db = _main.db_instance
689
690         notify = progress or (lambda stage: None)
691
692         notify("hashing")
693         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
694         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
695         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
696         # consumers (hash / validators / segmenter) use the resolved local path.
697         local_video = ""
698         try:
699             local_video = storage.acquire_local_input(
700                 req.video_path, getattr(config, "storage", None)
701             )
702             video_id = _main.compute_video_id(local_video)
703         except Exception:
704             storage.cleanup_acquired_local_input(
705                 req.video_path, local_video, getattr(config, "storage", None)
706             )
707             raise
708         try:
709             existing_video = db.get_video(video_id)
710         except Exception:
711             storage.cleanup_acquired_local_input(
712                 req.video_path, local_video, getattr(config, "storage", None)
713             )
714             raise
715         was_reingest = existing_video is not None
716
717         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
718         default_seg = getattr(
719             getattr(config, "indexing", None), "default_segmenter", "motion"
```

20. assistant (2026-07-07T15:50:21.052Z)

This is the critical lens. Let me verify how `google-cloud-run`'s proto-plus library actually exposes `completion_time` ? whether it's a proto `Timestamp` (with `.seconds`) or marshaled to a `datetime`. Let me check if the SDK is importable and inspect the actual proto.

21. user (2026-07-07T15:50:25.996Z)

```
run_v2 available
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\cloud\run_v2\__init__.py
---PROTO-PLUS---
proto-plus 1.28.0
```

22. assistant (2026-07-07T15:50:29.660Z)

The SDK is actually importable here. Let me construct an `Execution` proto and empirically check what `completion_time` is and whether `.seconds` works.

23. user (2026-07-07T15:50:37.923Z)

```
UNSET completion_time type: <class 'NoneType'> value: None
  getattr(ct,'seconds',0): 0
SET completion_time type: <class 'proto.datetime_helpers.DatetimeWithNanoseconds'> value:
DatetimeWithNanoseconds(2026, 7, 7, 12, 0, tzinfo=datetime.timezone.utc)
  getattr(ct2,'seconds',0): 0
GUARD on UNSET : returns None (treated as STILL RUNNING)
GUARD on SET   : returns None (treated as STILL RUNNING)
failed_count type: <class 'int'> getattr: 1
```

24. assistant (2026-07-07T15:51:35.277Z)

Confirmed empirically against the actual `google-cloud-run` proto-plus library. This is definitive and it's a serious defect.

Let me quickly rule out the secondary lens items (b)/(c)/(d) being independent problems, and confirm the count fields.

25. user (2026-07-07T15:51:42.937Z)

```
is datetime subclass: True
has 'seconds' attr: False | has 'second': True
classify(terminal-Failed exec) => None  (EXPECTED 'Failed')
```

26. assistant (2026-07-07T15:51:47.503Z)

Confirmed end-to-end: the real classifier returns `None` for a genuinely terminal-Failed execution, so the entire fast-fail mechanism is dead code in production.

Let me record the finding.

27. user (2026-07-07T15:52:13.544Z)

Structured output provided successfully